

# OnionMonad: Onion Circuits as Client-Sealed Loops of Registered Operation Symbols

Ryan J. Kung

## Abstract

The Rings onion data plane interprets two layers, relay and exit, and answers along the reversed path through relays that keep per-circuit return state and forward one client-bound ciphertext byte-identical. We generalise the circuit to a term over a static signature  $\Sigma$  of operation symbols: nodes are partial  $\Sigma$ -algebras that register symbols, never applications; a circuit is a client-sealed loop  $\diamond \rightarrow g \rightarrow \dots \rightarrow g \rightarrow \diamond$  that evaluates one application per hop and composes them in the Kleisli category of the hop effect monad, with **relay** as the identity. Each segment holds  $s = 2$  relays, the guard counting in both end segments; every hop parses one uniform layer of a fixed-length Sphinx-style header; values travel in a fixed-width carry slot enciphered with AEZ, one length-preserving layer per relay and one authenticated layer per consumer; replies to world-facing sessions ride single-use reply blocks under credit flow control. We state eleven laws, among them noninterference over hop views, position hiding, client path authority (which admits Arrow composition and excludes monadic bind), byte unlinkability, and single-use admission, list the residual leakage, and map each law to a test obligation.

## I. Introduction

Today's circuit is the term **relay** <sup>$k$</sup>  $\ggg(svc, body)$  evaluated on the forward path and answered on its reverse (§X). This paper states the general structure of which it is the degenerate instance, as definitions D1–D9 and laws L1–L10 and L5' of issue #834, numbered identically. Design lineage: Sphinx headers and wide-block payloads [1], reply blocks [2], [3], loops [4], AEZ [5], Kleisli semantics [6], generic effects [7], [8], and the Arrow/Selective hierarchy [9]–[11].

## II. Signature and Algebras

**Definition D1 (signature).**  $\Sigma$  is a finite, statically defined table of symbols; each  $f \in \Sigma$  has

$$\text{spec}(f) = (\text{name}, \text{Args}_f, \text{In}_f, \text{Out}_f, \\ W(f), L(f), \text{pos}(f), \text{quota}(f))$$

with *name* identifying the backend (e.g. `jev.typesafe`),  $\text{Args}_f$  client constants,  $\text{In}_f, \text{Out}_f$  carried types,  $W(f) \in \mathbb{N}$  the exact output width in bytes,  $L(f)$  the exact latency class,  $\text{pos}(f) \subseteq \{\text{Intermediate}, \text{WorldFacing}\}$ , and  $\text{quota}(f)$  the admission budget.  $W, L$  are protocol data. The identity symbol is

$$\text{relay} : X \rightarrow X, \quad L(\text{relay}) = 0;$$

**relay** has no  $W$ : it preserves the slot width  $C$  (L3).

Ryan J. Kung can be reached at [ryankung@ieee.org](mailto:ryankung@ieee.org). Manuscript created September 24, 2026. This document freezes the design of Rings issue #834 (rings-node, module onion) before its Phase 2 code; it is a specification, not shipped behaviour. The constants  $s = 2$  and  $n_{\max} = 4$  are those fixed in the issue discussion.

**Definition D2 (registration).** Node  $n$  is a partial  $\Sigma$ -algebra: a set  $\Sigma_n \subseteq \Sigma$  and, for each  $f \in \Sigma_n$ , a generic effect [7]

$$\llbracket f \rrbracket_n : \text{Args}_f \times \text{In}_f \rightarrow M \text{Out}_f,$$

$M$  the hop effect monad (failure, delay, world I/O).  $n$  publishes  $f \in \Sigma_n$ , never an application. Every published symbol, **relay** included, carries  $n$ 's process epoch  $e_n$ , drawn afresh at each process start: **relay** in `OnlineNodeDescriptor::capabilities` extended with  $e_n$ , every other  $f$  in one signed descriptor per symbol with  $e_n$  and  $\text{quota}(f)$ . Registration is upward closed:  $\Sigma_n \neq \emptyset \Rightarrow \text{relay} \in \Sigma_n$ , so a neighbour's registry entry does not reveal whether a relay is adjacent to a symbol hop.

A node is a handler [8] for the operations it registers; **WorldFacing** symbols (`tcp`, `https`) may hold a session, **Intermediate** symbols are one-shot.

**Definition D3 (application).** An application is  $(f, \bar{a})$ ,  $\bar{a} \in \text{Args}_f$ , typed  $(f, \bar{a}) : \text{In}_f \rightsquigarrow \text{Out}_f$ ; it is sealed in the layer of the hop that evaluates it and visible to no other hop.

## III. Pipelines

**Definition D4a (pipeline).** Pipelines are the morphisms of the free category  $\mathcal{F}(\Sigma)$  on the graph with types as vertices and applications as edges; composition is

$$\frac{\theta : X \rightsquigarrow Y \quad \vartheta : Y \rightsquigarrow Z}{\theta \ggg \vartheta : X \rightsquigarrow Z}.$$

$P = (f_1, \bar{a}_1) \ggg \dots \ggg (f_n, \bar{a}_n)$  is well formed iff

$$1 \leq n \leq n_{\max} = 4, \quad \forall k < n : \text{Out}_{f_k} = \text{In}_{f_{k+1}},$$

$$\forall k < n : \text{Intermediate} \in \text{pos}(f_k),$$

$$f_n \text{ is a WorldFacing session symbol} \Rightarrow n = 1.$$

Violations are rejected at construction. The bound  $n_{\max}$  is `MAX_ONION_LOOP_SYMBOLS`.

Definition D4b (Kleisli denotation). For  $\phi : X \rightarrow MY$ ,  $\psi : Y \rightarrow MZ$  in  $\mathbf{KI}(M)$  [6],  $\phi \ggg \psi = \mu_Z \circ M\psi \circ \phi$ . For hop assignment  $h_1..h_n$  with  $f_k \in \Sigma_{h_k}$ ,

$$\llbracket P \rrbracket(v_0) = (\llbracket f_1 \rrbracket_{h_1}(\bar{a}_1, -) \ggg \cdots \ggg \llbracket f_n \rrbracket_{h_n}(\bar{a}_n, -))(v_0),$$

the unique functor  $\mathcal{F}(\Sigma) \rightarrow \mathbf{KI}(M)$  extending the hop interpretations. The client consumes  $\llbracket P \rrbracket(v_0)$ .

Law L1 (identity).  $\llbracket \text{relay} \rrbracket = \eta$ , hence  $\text{relay} \ggg f = f = f \ggg \text{relay}$  on values. The loop builder factors through  $\text{nf}$ , which deletes  $\text{relay}$  applications:  $\text{Loop}(P) = \text{Loop}(\text{nf}P)$ , so  $\text{Loop}(\text{relay} \ggg P) \stackrel{d}{=} \text{Loop}(P)$  and every hop's view is unchanged.

Law L2 (associativity).  $(f \ggg g) \ggg h = f \ggg (g \ggg h)$  on values; immediate from the monad laws, and  $\mathcal{F}(\Sigma)$  has no bracketing.

$\text{nf}$  and L1 on views apply from Phase 2a. Phase 1 admits only  $\text{relay}^k \ggg (f, \bar{a})$  with  $\text{WorldFacing} \in \text{pos}(f)$ , which is today's circuit (§X).

#### IV. The Loop

Definition D4c, D5 (loop, segment). Let  $\diamond$  be the client and  $g$  its entry guard. Segment  $k \in [0, n]$  is  $s = 2$  relays  $r_{k,1}..r_{k,s}$  (`ONION_SEGMENT_RELAYS`) from producer  $p_k$  to consumer  $c_k$ ,

$$\begin{aligned} p_0 = \diamond, \quad p_k = h_k, \quad c_k = h_{k+1}, \quad c_n = \diamond, \\ \text{Loop}(P) : \diamond \rightarrow r_{0,1..s} \rightarrow h_1 \rightarrow r_{1,1..s} \rightarrow \cdots \\ \rightarrow h_n \rightarrow r_{n,1..s} \rightarrow \diamond. \end{aligned}$$

Positions are  $i \in [1, H]$ ; position  $i$  evaluates  $f_i$ , which is  $\text{relay}$  on every  $r_{k,j}$ .  $\text{Loop}(\text{nf}P)$  is the symbol word  $\text{relay}^s f_1 \text{relay}^s \cdots f_n \text{relay}^s$ .

Law L7 (guard closure).  $r_{0,1} = r_{n,s} = g$ : the guard is relay 1 of segment 0 and relay  $s$  of segment  $n$ . The other  $H - 2$  positions carry pairwise distinct DIDs, distinct from  $g$ .  $g$  is the only hop adjacent to  $\diamond$ .

Proposition 1 (Loop size). With  $n$  symbol hops and  $n + 1$  segments of  $s$  relays,

$$\begin{aligned} H(n, s) &= n + (n + 1)s = (s + 1)n + s, \\ \# \text{DID} &= H - 1, \quad E = H + 1 \text{ cells}, \\ \hat{H} &= H(n_{\max}, s) = (s + 1) \cdot 4 + s = 14. \end{aligned}$$

Proof. Segments contribute  $(n + 1)s$  positions; L7 repeats exactly one DID; the  $H$  positions lie on a path from  $\diamond$  to  $\diamond$  of  $H + 1$  edges.  $\square$

Proposition 2 (Segment length).  $s = 2$  is the least  $s$  at which no single node is adjacent to both  $\diamond$  and a symbol hop.

Proof.  $s = 0$  violates L7. At  $s = 1$ ,  $r_{0,1} = r_{0,s} = r_{n,1} = r_{n,s} = g$ , so  $g$  alone is adjacent to  $\diamond$ ,  $h_1$  and  $h_n$ , and for

Table I  
Loop shape at  $s = 2$  (Prop. 1)

| $n$ | $H$ | DIDs | $E$ | header $ \beta $             |
|-----|-----|------|-----|------------------------------|
| 1   | 5   | 4    | 6   | $\hat{H}\ell = 9086\text{B}$ |
| 2   | 8   | 7    | 9   |                              |
| 3   | 11  | 10   | 12  |                              |
| 4   | 14  | 13   | 15  |                              |

$n = 1$  links  $\diamond$  to its world-facing  $h_1$ . At  $s = 2$ ,  $g$  neighbours only  $\diamond$ ,  $r_{0,2}$ ,  $r_{n,1}$ .  $\square$

Relays do not mix ( $L(\text{relay}) = 0$ ), so a timing-correlating coalition skips middle relays:  $s \geq 3$  adds  $n + 1$  edges of latency per relay and no anonymity. At  $(s, n) = (2, 1)$ ,  $E = 6$  and the forward prefix  $g, r_{0,2}, h_1$  is Tor's three-hop circuit [12].

Route validation checks  $H = (s + 1)n + s$  with exactly  $s$  relays between consecutive symbol hops, and admits  $g$  exactly twice, at positions 1 and  $H$  (has `duplicate_dids`); selection fails closed when fewer than  $H - 1$  distinct eligible DIDs exist. `MAX_ONION_LOOP_HOPS` =  $\hat{H}$  is derived from  $s$  and  $n_{\max}$  and replaces `MAX_ONION_CIRCUIT_HOPS` = 8; `DEFAULT_ONION_ROUTE_HOPS` and `OnionRouteRequest::{\code{hop_count}, \code{allow_short_paths}}` are removed, since a short path is a distinguishable segment length (D5).

#### V. Uniform Layer and Timing

Definition D6 (uniform layer). Every position  $i$  has one layer shape

$$\lambda_i = (f_i, \bar{a}_i, \text{next}_i, e_i, x, \nu_i, \kappa_i, \gamma_{i+1}, \alpha_{i+1})$$

of  $\ell$  bytes, with  $\bar{a}_i$  padded to  $A$  bytes: epoch  $e_i$ , one expiry  $x$  per loop (a multiple of  $Q = 30\text{s}$ ), replay nonce  $\nu_i$ , carry keys  $\kappa_i = (k_i^{\text{in}}, k_{i,1..s+1}^{\text{out}})$ , and the next hop's MAC  $\gamma_{i+1}$  and key transport  $\alpha_{i+1}$  (Def. 1). A relay has  $\bar{a}_i = ()$  and random  $k^{\text{out}}$ . Each loop has one class  $b \in \mathfrak{B}$ , today's cell buckets above 4 KiB (16 KiB to 12 MiB), chosen by  $\diamond$ ; the cell on every edge of the loop, SURB replies included, is

$$(\chi_{i+1}, y_i), \quad \chi = (\alpha, \beta, \gamma), \quad |\beta| = \hat{H}\ell, \quad |y_i| = C_b,$$

padded to  $b$ , with  $C_b = b - |\chi| - F$ .

With today's encodings (239-byte ElGamal-AEAD envelope as  $\alpha$ , 43-byte DIDs, 52-byte public keys) and 48-byte AEZ keys,  $\ell(s, A) = 393 + A + 48(s + 2)$ , so  $\ell(2, 64) = 649\text{B}$  and  $|\beta| = 9086\text{B}$ : every loop cell uses at least the 16 KiB bucket.

Definition 1 (Fixed-length header). Hop  $i$  derives  $K_i$  from  $\alpha_i$  and its secret key and expands  $\rho_i = \text{PRG}(K_i) \in \{0, 1\}^{(\hat{H}+1)\ell}$ ;  $\rho[a, b]$  is a byte range in units of  $\ell$ . It checks  $\gamma_i = \text{MAC}_{K_i}(\beta_i)$  and peels

$$(\beta_i \parallel 0^\ell) \oplus \rho_i = \lambda_i \parallel \beta_{i+1}.$$

The client builds, for  $1 \leq i < H$ , with  $R$  uniform,

$$\begin{aligned} \phi_0 &= \epsilon, \quad \phi_i = (\phi_{i-1} \| 0^\ell) \oplus \rho_i[\hat{H}-i+1, \hat{H}+1), \\ \beta_H &= ((\lambda_H \| R) \oplus \rho_H[0, \hat{H}-H+1)) \| \phi_{H-1}, \\ \beta_i &= (\lambda_i \| \beta_{i+1}[0, \hat{H}-1)) \oplus \rho_i[0, \hat{H}), \quad \gamma_i = \text{MAC}_{K_i}(\beta_i). \end{aligned}$$

This is Sphinx's header [1]: per-hop PRG filler with MACs the client precomputes.  $\alpha_{i+1}$  is either a per-layer envelope carried in  $\lambda_i$  (as above) or a blinding of  $\alpha_i$  (Sphinx's single group element, roughly halving  $\ell$ ); L5' holds for both.

Proposition 3 (Header correctness). For  $H \leq \hat{H}$  and  $1 \leq i < H$ , peeling  $\beta_i$  under  $K_i$  yields  $(\lambda_i, \beta_{i+1})$ , peeling  $\beta_H$  yields  $\lambda_H$ , and every  $\gamma_i$  verifies.

Proof. By induction down from  $H$ , the last  $i-1$  blocks of  $\beta_i$  are  $\phi_{i-1}$ : true at  $H$  by construction, and expanding  $\beta_i$  and  $\phi_i$  carries it from  $i+1$  to  $i$ . So the first  $\hat{H}$  blocks of  $(\beta_i \| 0^\ell) \oplus \rho_i$  are  $\lambda_i \| \beta_{i+1}[0, \hat{H}-1)$ , and its last block  $\rho_i[\hat{H}, \hat{H}+1)$  is the last block of  $\phi_i$ , hence of  $\beta_{i+1}$ .  $\square$

Law L3 (width). The carry slot has width  $C = C_b$  on every edge of a loop of class  $b$ . With  $C_0 = C - \tau$ , the client picks the least  $b$  with  $W(f_k) < C_0$  for all  $k$ .  $W$  factors through the output type ( $W(f) = w(\text{Out}_f)$ , so the width a consumer un pads is fixed by its own In); a WorldFacing stream symbol (tcp, https) has  $\text{In} = \text{Out} = \text{Bytes}_{<C_0}$ , one chunk per loop or reply, and larger bodies span several loops. Further,

$$\text{pad}(b) = b \| 80 \| 00^{C_0-|b|-1}, \quad |\text{enc}_f(w)| = W(f);$$

an output with  $|\text{enc}_f(w)| > W(f)$  fails closed, never truncated.

Law L4 (timing).  $t_i^{\text{dep}} = t_i^{\text{arr}} + L(f_i)$  exactly. An evaluation unfinished at  $t_i^{\text{dep}}$  is dropped, never released late.

## VI. Carry

Write  $E_k^\tau, D_k^\tau$  for AEZ v5 [5], [13] encryption and decryption with empty nonce and associated data;  $|E_k^\tau(m)| = |m| + \tau$ , and  $D_k^\tau$  returns  $\perp$  unless the deciphered block ends in  $\tau$  zero bytes.

Definition D7 (carry). For segment  $k$  the client draws fresh keys  $k_{r_{k,1}}, \dots, k_{r_{k,s}}, k_{c_k}$  and places them

$$\kappa_{p_k}^{\text{out}} = (k_{r_{k,1}}, \dots, k_{r_{k,s}}, k_{c_k}), \quad \kappa_{r_{k,j}}^{\text{in}} = k_{r_{k,j}}, \quad \kappa_{c_k}^{\text{in}} = k_{c_k}.$$

With  $b_k = \text{pad}(\text{enc}(v_k))$  and  $\tau = 16$ :

$$\begin{aligned} y_{k,0} &= E_{k_{r_{k,1}}}^0 \circ \dots \circ E_{k_{r_{k,s}}}^0 \circ E_{k_{c_k}}^\tau(b_k), \\ y_{k,j} &= D_{k_{r_{k,j}}}^0(y_{k,j-1}), \quad b_k = D_{k_{c_k}}^\tau(y_{k,s}). \end{aligned}$$

A value of type  $()$  is encoded as  $\epsilon$ , so every consumer authenticates  $\text{pad}(\epsilon)$  and processing is uniform.

Since  $s$  is constant,  $|\kappa_i|$  is constant and reveals no position. AEZ v5 is implemented in-repo on RustCrypto aes::hazmat (constant-time, wasm32); keys are raw 48-byte strings, a key with a zero subkey  $I, J$  or  $L$  is rejected,

Algorithm 1 Hop<sub>i</sub>(from,  $\chi, y, t^{\text{arr}}$ ): one position, all symbols

---

```

1:  $(\lambda, \chi') \leftarrow \text{peel}(sk_i, \chi)$ ; if  $\gamma$  fails then drop
2: if  $x \notin (t^{\text{arr}}, t^{\text{arr}} + V] \vee e \neq \text{epoch}_i \vee \nu \in R_i \vee R_i^{\text{from}}$  full then
3:   drop ▷ L9
4:  $R_i \leftarrow R_i \cup \{(\nu, t^{\text{arr}} + V)\}$ 
5: if  $f = \text{relay}$  then
6:    $y' \leftarrow D_{k_{\text{in}}}^0(y)$ 
7: else
8:    $b \leftarrow D_{k_{\text{in}}}^\tau(y)$ ; if  $b = \perp$  then drop ▷ L8
9:   if  $f$  is a WorldFacing session symbol then
10:     $Q_h \leftarrow Q_h \cup \{(\text{next}, \chi', \kappa^{\text{out}}, x)\}$  ▷ D8
11:    write  $\text{pad}^{-1}(b)$  to the world; return
12:    $w \leftarrow \llbracket f \rrbracket(\bar{a}, \text{enc}^{-1} \text{pad}^{-1}(b))$  by  $t^{\text{arr}} + L(f)$  ▷ L4
13:   if late  $\vee |\text{enc}_f(w)| > W(f)$  then
14:     drop ▷ L3
15:    $y' \leftarrow E_{k_1^{\text{out}}}^0 \circ \dots \circ E_{k_s^{\text{out}}}^0 \circ E_{k_{s+1}^{\text{out}}}^\tau(\text{pad enc}_f(w))$ 
16: release  $(\chi', y')$  to next at  $t^{\text{arr}} + L(f)$ 

```

---

Algorithm 2 Reply<sub>h</sub>( $t$ ): world bytes ready at session hop  $h$

---

```

1:  $Q_h \leftarrow \{v \in Q_h : t < x_v\}$  ▷ expired SURBs discarded
2: if  $Q_h = \emptyset$  then
3:   pause reading; return ▷ Inv. 1
4: take  $v = (\text{next}, \chi, k_1..k_{s+1}, x) \in Q_h$  of least  $x$ 
5:  $w \leftarrow$  at most  $C_0 - 1$  world bytes
6:  $y' \leftarrow E_{k_1}^0 \circ \dots \circ E_{k_s}^0 \circ E_{k_{s+1}}^\tau(\text{pad}(w))$ 
7: release  $(\chi, y')$  to next

```

---

and  $\text{GF}(2^{128})$  doubling is branch-free.  $C \geq 32$  keeps every call on the AEZ-core path, never AEZ-tiny.

Proposition 4 (One value per key). Under L9 every  $k_{r_{k,j}}$  and  $k_{c_k}$  is applied to exactly one  $C$ -byte value: enciphered once by  $p_k$ , deciphered once by its holder.

Proof. Keys are fresh per layer (D7).  $p_k$  enciphers its single output once under each key of  $\kappa_{p_k}^{\text{out}}$ . The holder of  $k$  decipheres only a cell whose layer names  $k$ , and L9 admits that layer once, across restarts by the process epoch. Without L9 a replayed cell with a modified carry makes the holder a chosen-ciphertext oracle under a fixed key: L9 is load-bearing for AEZ.  $\square$

Definition 2 (Client position).  $\diamond$  is position  $H+1$ .  $\lambda_H$  carries, in place of  $\alpha_{H+1}$ , a fresh tag  $t_\diamond \in \{0,1\}^{128}$ ; the guard sends  $(\chi_{H+1}, y_H)$  with  $\chi_{H+1} = (t_\diamond, \beta_{H+1}, \gamma_{H+1})$ .  $\diamond$  keeps  $T : t_\diamond \mapsto (k_{c_n}, \text{loop})$ , removes the entry on first use, and drops a cell with an unknown tag. Every SURB suffix ends in its own tag.

$t_\diamond$  is uniform and seen only by  $g$  at position  $H$  and by  $\diamond$ , so it links nothing the guard cannot already link by timing.

## VII. Reply Blocks

Definition D8 (SURB). A single-use reply block for a session symbol at  $h$  is the client-sealed loop suffix after  $h$ :

$$v = (\text{next}, \chi_v, k_{r_1}, \dots, k_{r_s}, k_\diamond, x_v).$$

$h$  keeps the suffix of every forward loop that reaches it ( $\chi_{i+1}, \kappa_i^{\text{out}}, x$ ) as one SURB; no SURB rides in a carry value.

Proposition 5 (No SURB in the carry). A carried SURB forces  $C > \hat{H}\ell$  on every edge.

Proof. L5' fixes  $|\chi_v| = |\chi| > |\beta| = \hat{H}\ell$  (a shorter reply header would name the reply path), and L3 fixes one  $C$  for every edge of a loop and its replies.  $\square$

At  $s = 2$  that would at least double every cell (9086 B of  $\beta$  in the carry besides the header's own), so credit arrives only as forward loops: one forward cell per reply cell, data optional.

Invariant 1 (Credit). Let  $Q_h$  be  $h$ 's unexpired SURBs ( $t_v$  the arrival of  $v$ ),  $u(t)$  the forward loops of the session received and  $r(t)$  the replies emitted by time  $t$ . Then

$$r(t) \leq u(t), \quad v \in Q_h \Rightarrow t < x_v \leq t_v + V, \\ |Q_h| = 0 \Rightarrow \text{tcp stops reading its socket.}$$

Each reply consumes one  $v$ , produced as segment  $n$  of D7 with  $p_n = h$ ; an expired  $v$  is discarded unused; a new forward loop resumes reading. A session idle for longer than  $V$  holds no credit.

## VIII. Security Laws

Assumption 1 (Primitives). Under a key it does not hold, a hop sees  $E_k^0$  as a uniformly random permutation of  $\{0, 1\}^C$  and  $E_k^r$  as robust AE [5]; PRG and MAC are secure, and key transport reveals  $K_j$  to hop  $j$  only. Network delays  $\delta$  are fixed per edge; no hop other than  $g$  observes the client's departure time.

Definition D9 (view). Over client coins  $\rho$  (keys, nonces, header randomness) and delays  $\delta$ ,

$$\text{view}_i = (f_i, \bar{a}_i, v_{i-1}, \text{from}_i, \text{next}_i, e_i, x, b, \\ \text{bucket}, t_i^{\text{arr}}, t_i^{\text{dep}}, \chi_i, y_{i-1}, \chi_{i+1}, y_i),$$

with  $v_{i-1} = \perp$  when  $f_i = \text{relay}$  (a relay consumes no value).

Law L5 (noninterference). For loops  $P, P'$  let

$$\pi_i(P) = (f_i, \bar{a}_i, v_{i-1}, \text{from}_i, \text{next}_i, e_i, x, b), \\ \sigma(P) = f_1 \cdots f_H, \\ \pi_i(P) = \pi_i(P') \wedge \sigma(P) = \sigma(P') \\ \Rightarrow \text{view}_i(P) \stackrel{d}{=} \text{view}_i(P').$$

Symbols may leak;  $\bar{a}_j, v_j$  for  $j \neq i$ , other than the consumed  $v_{i-1}$ , may not [14].

Proposition 6 (L5 from D6, D7, L3, L4). Under Assumption 1, every loop built by D4c–D7 satisfies L5.

Proof. Lengths:  $|\chi_i|$  and  $|y_i| = C$  are constant (D6, L3). Times:  $t_i^{\text{arr}} = \sum_{j < i} (L(f_j) + \delta_j)$  by L4, a function of  $\sigma, \delta$ . Headers:  $\chi_i, \chi_{i+1}$  reveal only  $\lambda_i$ , fixed by  $\pi_i$  up to fresh coins (Prop. 7). Carry: if  $i = r_{k,j}$ , then  $D_{k_{r_{k,j}}}^0(y_{k,j-1})$  is an E output under  $k_{r_{k,j+1}}$  or  $k_{c_k}$ , a fresh key not held by  $i$ , hence uniform on  $\{0, 1\}^C$  and independent of  $v_k$ . If  $i = c_k$ , it learns  $v_{i-1}$ , which  $\pi_i$  fixes. If  $i = p_k$ , its output is computed from  $v_{i-1}$  and  $\bar{a}_i$ . No component depends on  $\bar{a}_j, v_j$  otherwise.  $\square$

L5's premise fixes  $\sigma$ , hence  $H$  and every index, so it says nothing about length or position; L5' does.

Law L5' (position hiding). For loops  $P, P'$  with  $H(P), H(P') \leq \hat{H}$  and positions  $i$  of  $P$ ,  $i'$  of  $P'$ ,

$$\pi_i^\circ(P) = (f_i, \bar{a}_i, v_{i-1}, \text{from}_i, \text{next}_i, b), \\ \pi_i^\circ(P) = \pi_{i'}^\circ(P') \Rightarrow \text{view}_i(P) \approx_c \text{view}_{i'}(P').$$

No hop learns its position,  $H$ , or the symbols elsewhere in the loop, except the guard:  $\text{from} = \diamond$  or  $\text{next} = \diamond$  names position 1 or  $H$ , and  $\pi^\circ$  records it.

Proposition 7 (L5' from Def. 1). Under Assumption 1, the distinguishing advantage is at most  $\hat{H} \epsilon_{\text{PRG}} + \epsilon_{\text{MAC}} + \epsilon_\alpha$ .

Proof.  $|\chi|$  and  $C$  do not depend on  $H$  or  $i$ ;  $e_i, x$  and  $t_i^{\text{arr}}$  carry no position (one expiry per loop; departure time unobserved). Hop  $i$  sees  $\lambda_i$ , fixed by  $\pi_i^\circ$  up to fresh coins,  $\alpha_{i+1}, \gamma_{i+1}$ , and  $\beta_{i+1}$ . Every block of  $\beta_{i+1}$  but the last is masked by some  $\rho_j, j \neq i$ : by  $\rho_{i+1}$  when  $i+1 < H$ , and by  $\rho_H$  or an upstream term of  $\phi_{H-1}$  when  $i+1 = H$ . The last block is  $\rho_i[\hat{H}, \hat{H}+1)$ , computed by  $i$  itself in both loops. Hybrids replacing the  $\rho_j, j \neq i$ , one key at a time by uniform strings make  $\beta_{i+1}$  uniform apart from that block, in  $P$  and  $P'$  alike;  $\gamma_{i+1}$  is then a MAC under an unseen key.  $\square$

Law L6 (path authority).  $\text{next}_i$  is a field of  $\lambda_i$ , fixed by  $\diamond$  at construction:  $\text{next}_i$  is independent of every  $v_j$ . No symbol chooses a hop.

Proposition 8 (Admissible combinators). L6 admits the composition fragment ( $\gg$ , without *arr* or *first*) of Arrows [9] and excludes monadic bind. Selective branching [11] over client-sealed branches needs L6 weakened to  $\text{next}_i \in N_i$ , a finite set sealed by  $\diamond$ .

Proof.  $\gg$  fixes every hop before any value exists. *select* :  $F(A + B) \rightarrow F(A \rightarrow B) \rightarrow FB$  fixes the possible effects, so  $\diamond$  can seal both branches, but the selecting hop picks  $\text{next}_i$  by  $v_i$  and learns the branch taken; its law is deferred with Selective itself. Bind  $m \gg k, k : A \rightarrow MB$ , chooses the continuation, and so its hops, from a run-time value: either a hop constructs  $\text{next}$  (violating L6) or  $\diamond$  seals  $|A|$  continuations. This is the arrows-meticulous, monads-promiscuous separation of [10].  $\square$

“OnionMonad” names  $M$ , the effect inside one hop; across hops only the static fragment of  $\mathbf{KI}(M)$  is exposed.

Law L8 (unlinkability). On consecutive edges  $\chi_i \neq \chi_{i+1}$  and  $y_{i-1} \neq y_i$  (one layer peeled, one AEZ layer removed per hop). If any  $y_{k,j}$  is modified to  $\tilde{y} \neq y_{k,j}$ ,

$$\Pr[D_{k_{c_k}}^\tau(\tilde{y}_{k,s}) \neq \perp] \leq 2^{-8\tau} + \epsilon_{\text{AEZ}},$$

and a rejected value yields nothing; no bit of a tag survives. L8 is a byte-level law; colluders still link by timing (Table II).

Under Assumption 1 consecutive  $y$  are independent uniform strings to any hop lacking both keys. A modification propagates through every later wide-block decipherment, so, unlike counter-mode relay cryptography [15], a flipped bit is never observed in place; this is Sphinx’s wide-block payload [1], [16] with AEZ in place of LIONESS.

Law L9 (replay). With  $V = 150$  s (ONION\_FORWARD\_MAX\_VALIDITY\_MS) and per-sender capacity  $B = 4096$  over at most 64 senders, hop  $i$  admits a layer at  $t^{\text{arr}}$  iff

$$t^{\text{arr}} < x \leq t^{\text{arr}} + V \wedge \nu \notin R_i \wedge |R_i^{\text{from}}| < B,$$

then retains  $\nu$  in  $R_i$  until  $t^{\text{arr}} + V$ . A full  $R_i^{\text{from}}$  rejects; no tag leaves  $R_i$  before its retention ends. A SURB is a layer, hence single use; L9 also bounds each AEZ key to one carry value (Prop. 4).

Proposition 9 (Replay bounds). (i) No admitted layer outlives its tag:  $x \leq t^{\text{arr}} + V$ . (ii)  $|R_i| \leq 64B$ . (iii) Hop  $i$  admits at most  $B$  layers per sender per  $V$ , i.e.  $B/V \approx 27$  cells  $\text{s}^{-1}$  per neighbour. (iv) A SURB is usable only within  $[x_v - V, x_v)$ , so SURB lifetime is at most  $V$ .

Proof. (i) is the admission window; (ii) the capacity guard; (iii) each admitted tag occupies a slot for  $V$ ; (iv) every relay of the reply path applies (i) at the reply’s arrival.  $\square$

(iii) bounds per-neighbour loop throughput, including SURB replies;  $B$  is a tuning constant, not a law.

Law L10 (wire). Phase 1 preserves today’s bytes (golden tests); Phase 2 replaces them with a new pinned golden set in one cutover, without versioning.

## IX. Accepted Leakage

Table II is exhaustive with respect to L5: every other component of  $\text{view}_i$  is determined by  $\pi_i$  and  $\sigma$ .

## X. The Current Design

Proposition 10 (Backward linkability). Today the exit seals its answer once,  $c = \text{AEAD}_{pk_\circ}(\cdot)$  (encrypt\_client\_payload\_at\_sequence in circuit/crypto.rs), and each relay re-emits it as  $\text{payload} = \text{frame.payload}$  (advance\_backward in circuit/reducer.rs), rewriting only the edge circuit id and the outer hop seal. Every backward relay therefore sees the same  $c$ : any two colluding relays, and the exit, link their positions by equality, independently of route length.

Proposition 11 (Nested layers leak position). build\_forward\_layers\_with\_ids nests one envelope per hop, so

Table II  
Residual leakage, stated and accepted

| leakage                                                                            | bound                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| symbols $\sigma$ are public;                                                       | L5 hypothesis                                                                                                                                                                                         |
| $\lambda_i$ names $f_i$<br>a WorldFacing hop<br>knows it faces the<br>world        | no other hop learns whether<br>symbols follow (L3, D6, L5')                                                                                                                                           |
| $g$ sees departure and<br>return                                                   | round-trip $\sum_e d_e + \sum_i L(f_i)$ : an<br>estimate of $H$ and of $\sum_i L(f_i)$ ;<br>normalising it is a separate issue                                                                        |
| latency class                                                                      | L4 quantises timing; cannot<br>remove it                                                                                                                                                              |
| hosted backend                                                                     | sees $\bar{a}$ of every application of its<br>symbol                                                                                                                                                  |
| loop class $b$                                                                     | every hop sees it; the same<br>leakage as today’s cell buckets<br>shared by all hops of a loop and<br>all loops sealed in one $Q = 30$ s<br>quantum: colluders link positions<br>only within that set |
| expiry $x$                                                                         | $p_k$ colluding with $r_{k,j}$ links two<br>positions of one segment; no<br>value leaks, and timing links<br>them anyway                                                                              |
| $p_k$ holds $\kappa_{p_k}^{\text{out}}$                                            | out of protocol scope                                                                                                                                                                                 |
| client-authored<br>semantics ( $\bar{a}_{i+1}$<br>depends on meaning of<br>$v_i$ ) |                                                                                                                                                                                                       |

Table III  
Current circuit versus OnionMonad

| concern            | current                                                         | here                              |
|--------------------|-----------------------------------------------------------------|-----------------------------------|
| layers             | Relay, Exit                                                     | one uniform $\lambda_i$ (D6)      |
| header             | nested, $-404$ B per hop                                        | fixed $H\ell$ (L5')               |
| length             | 3 by default, $\leq 8$ , short<br>paths allowed                 | $H(n, 2) \leq 14$ (D5)            |
| computation        | exit service only                                               | any $f \in \Sigma_n$ (D2)         |
| return path        | forward path reversed                                           | fresh relays to $g$ (L7)          |
| relay state        | relay_returns, $\leq 1024$ ,<br>TTL 120 s refreshed per<br>cell | replay tags only (L9)             |
| return bytes       | one ciphertext, identical<br>per edge                           | one AEZ layer per edge<br>(L8)    |
| replay             | exits (consume_forward_<br>nonce), $V = 150$ s,<br>$B = 4096$   | every hop, same $V$ , $B$<br>(L9) |
| session<br>replies | no credit bound                                                 | one per SURB (Inv. 1)             |

hop  $i$  decrypts  $\text{inner}_i$  with  $|\text{inner}_i| = \sum_{j>i} \ell_j + |\lambda_H|$ , measured at 404 B per downstream hop. A loop extended after  $i$  changes  $|\text{inner}_i|$ , so  $\text{view}_i$  distinguishes it with probability 1: L5' fails.

Cell buckets pad the cell before decryption, not the layer; padding the innermost layer to a constant total only moves the leak from  $H - i$  to  $i$ .

The stored return edge  $(id', \text{next}) \mapsto (\text{from}, id, pk)$  is the only reason relays hold circuit state; the loop removes it with Backward, OnionBackwardFrame, MAX\_ONION\_RELAY\_CIRCUITS, and ONION\_RELAY\_RETURN\_TTL\_MS.

Table IV  
Threats and the law that bounds each

| threat                                | bound                                                                        |
|---------------------------------------|------------------------------------------------------------------------------|
| hop learns another's $\bar{a}_j, v_j$ | L5 (Prop. 6)                                                                 |
| position, loop length                 | fixed $ \beta , C,  \kappa , s$ (D5–D7, L3, L5')                             |
| value-dependent timing                | exact release $t^{\text{arr}} + L(f)$ (L4)                                   |
| colluding relays                      | per-edge re-encipherment (L8), up to the shared $x$ (Table II); cf. Prop. 10 |
| match bytes                           | whole-value rejection, $2^{-8\tau}$ (L8)                                     |
| tagging                               | next $_i \in \lambda_i$ (L6); bind excluded (Prop. 8)                        |
| hop-chosen routing                    | one guard closes the loop (L7)                                               |
| first/last-hop correlation            | single admission per $\nu$ (L9)                                              |
| replay of layer or SURB               | new process epoch rejects every older layer (D2, L9)                         |
| replay across a restart               | $r(t) \leq u(t)$ (Inv. 1)                                                    |
| reply amplification                   | per-symbol quota( $f$ ) (D1)                                                 |
| symbol abuse                          | fail closed at $W(f)$ (L3)                                                   |
| oversized result                      |                                                                              |

## XI. Security

Table IV maps each threat to the law that bounds it.

The adversary controls any set of nodes other than  $\diamond$ , and delivery order; it cannot break Assumption 1. Relays do not mix, so this is Tor's threat model: a global passive observer is out of scope. Dropping is always possible and out of scope.

## XII. Verification

Every law is discharged against the pure builder and reducer, not a shadow model (Table V).

## XIII. Phases

Table VI orders the cutover. Fixed:  $s = 2, n_{\max} = 4, \hat{H} = 14$ , in-repo AEZ. Open: key transport (per-layer envelope or one blinded element), raw DID and key encodings ( $\ell(2, 64) \approx 531$  B), and the replay capacity  $B$ . Non-goals: bind, Selective branches before Phase 2b, session symbols in intermediate position, concrete symbols, and verification of results.

## XIV. Related Work

Tor [12] answers along the circuit it built, as Rings does today; Degabriele and Stam [15] formalise the tagging that its counter-mode relay cryptography admits. Chaum's untraceable return addresses [2] and Mixminion's SURBs [3] are the source of D8; Sphinx [1] gives the fixed-length header and a LIONESS [16] payload whose zero-block check L8 inherits. Loopix [4] routes traffic as loops for cover; here the loop is the circuit. AEZ [5], [13] supplies both the length-preserving relay layer and robust AE in one primitive. On the semantic side, Moggi's Kleisli semantics [6], generic effects and handlers [7], [8], and the Arrow/Selective/Monad hierarchy [9]–[11] fix which compositions a client can seal in advance.

Table V  
Obligations and the test that discharges each

| obligation | test or property                                                                                                                                                                                                                      |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| L10        | golden encodings of layers, wire messages, cells, and descriptor signing data, landed before any refactor; a new pinned set at the Phase 2 cutover                                                                                    |
| L1         | carry through <b>relay</b> is identity;<br>$\text{Loop}(\text{relay} \gg P) \stackrel{d}{=} \text{Loop}(P)$                                                                                                                           |
| D4a        | <b>Out</b> $\neq$ <b>In</b> , <b>WorldFacing</b> before the last symbol, empty pipeline: rejected at construction                                                                                                                     |
| L3         | fixed slot width on every edge; output $> W(f)$ dropped                                                                                                                                                                               |
| L4         | release at $t^{\text{arr}} + L(f)$ and drop on overrun under an injected clock                                                                                                                                                        |
| L5         | property test on the pure builder: vary $\bar{a}_j, v_j, j \neq i$ ; view $_i$ bytes, lengths, and release times equal                                                                                                                |
| L5'        | builder for $H \neq H' \leq \hat{H}$ : equal layer and cell lengths at every position; every $\gamma_i$ verifies (Prop. 3)                                                                                                            |
| L3, D6     | one class $b$ and slot width $C_b$ on every edge of a loop, SURB replies included; stream chunks $< C_0$                                                                                                                              |
| client     | reply accepted once by tag $t_0$ ; unknown or reused tag dropped                                                                                                                                                                      |
| D5, L7     | $H = (s + 1)n + s$ ; guard at 1 and $H$ only; $n > 4$ rejected; selection fails closed below $H - 1$ eligible DIDs                                                                                                                    |
| L8         | per-hop byte inequality; one-bit flip rejects the whole value                                                                                                                                                                         |
| L9         | replayed layer and reused SURB rejected; a layer for an earlier process epoch rejected after restart; $x \notin (t^{\text{arr}}, t^{\text{arr}} + V]$ rejected; full $R_i^{\text{from}}$ rejects without evicting a live tag          |
| credit     | a tcp session emits no reply without a SURB, drops expired SURBs, and resumes after a new forward loop                                                                                                                                |
| AEZ        | 2941 vectors from the AEZ v5 reference C code (length sweep, parameter grid, CAESAR genkat), bit-flip rejection for $\tau \geq 4$ , native and wasm32; weak-key rejection; fuzz/ target for $D \circ E = \text{id}$ and panic freedom |

## References

- [1] G. Danezis and I. Goldberg, "Sphinx: A compact and provably secure mix format," in Proceedings of the 30th IEEE Symposium on Security and Privacy. IEEE, 2009, pp. 269–282.
- [2] D. L. Chaum, "Untraceable electronic mail, return addresses, and digital pseudonyms," Communications of the ACM, vol. 24, no. 2, pp. 84–90, 1981.
- [3] G. Danezis, R. Dingledine, and N. Mathewson, "Mixminion: Design of a type III anonymous remailer protocol," in Proceedings of the 2003 IEEE Symposium on Security and Privacy. IEEE, 2003, pp. 2–15.
- [4] A. M. Piotrowska, J. Hayes, T. Elahi, S. Meiser, and G. Danezis, "The Loopix anonymity system," in Proceedings of the 26th USENIX Security Symposium. USENIX Association, 2017, pp. 1199–1216.
- [5] V. T. Hoang, T. Krovetz, and P. Rogaway, "Robust authenticated-encryption: AEZ and the problem that it solves," in Advances in Cryptology – EUROCRYPT 2015, ser. Lecture Notes in Computer Science, vol. 9056. Springer, 2015, pp. 15–44.

Table VI  
Cutover plan

| phase | content                                                                                                                                                                     |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | golden bytes; $\Sigma$ table; pipeline term; layer build as a fold over $P$ ; reducer and handler as $\Sigma$ -algebra; route by symbol sequence. Zero wire change, no bump |
| 2a    | uniform layer, fixed-length header ( $L5'$ ), loop with derived $\hat{H}$ , loop classes, relay process epoch, client tag, stateless relays, AEZ carry, SURBs. Breaking     |
| 2b    | intermediate symbols: Invoke with release time, $\text{quota}(f)$ . Same cutover; one version bump on its last PR                                                           |

- [6] E. Moggi, “Notions of computation and monads,” *Information and Computation*, vol. 93, no. 1, pp. 55–92, 1991.
- [7] G. Plotkin and J. Power, “Algebraic operations and generic effects,” *Applied Categorical Structures*, vol. 11, no. 1, pp. 69–94, 2003.
- [8] G. Plotkin and M. Pretnar, “Handlers of algebraic effects,” in *Programming Languages and Systems (ESOP 2009)*, ser. Lecture Notes in Computer Science, vol. 5502. Springer, 2009, pp. 80–94.
- [9] J. Hughes, “Generalising monads to arrows,” *Science of Computer Programming*, vol. 37, no. 1–3, pp. 67–111, 2000.
- [10] S. Lindley, P. Wadler, and J. Yallop, “Idioms are oblivious, arrows are meticulous, monads are promiscuous,” *Electronic Notes in Theoretical Computer Science*, vol. 229, no. 5, pp. 97–117, 2011.
- [11] A. Mokhov, G. Lukyanov, S. Marlow, and J. Dimino, “Selective applicative functors,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. ICFP, pp. 90:1–90:29, 2019.
- [12] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The second-generation onion router,” in *Proceedings of the 13th USENIX Security Symposium*. USENIX Association, 2004, pp. 303–320.
- [13] V. T. Hoang, T. Krovetz, and P. Rogaway, “AEZ v5: Authenticated encryption by enciphering,” *CAESAR competition submission*, third round, 2017.
- [14] J. A. Goguen and J. Meseguer, “Security policies and security models,” in *Proceedings of the 1982 IEEE Symposium on Security and Privacy*. IEEE, 1982, pp. 11–20.
- [15] J. P. Degabriele and M. Stam, “Untagging Tor: A formal treatment of onion encryption,” in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10822. Springer, 2018, pp. 259–293.
- [16] R. Anderson and E. Biham, “Two practical and provably secure block ciphers: BEAR and LION,” in *Fast Software Encryption (FSE 1996)*, ser. Lecture Notes in Computer Science, vol. 1039. Springer, 1996, pp. 113–120.